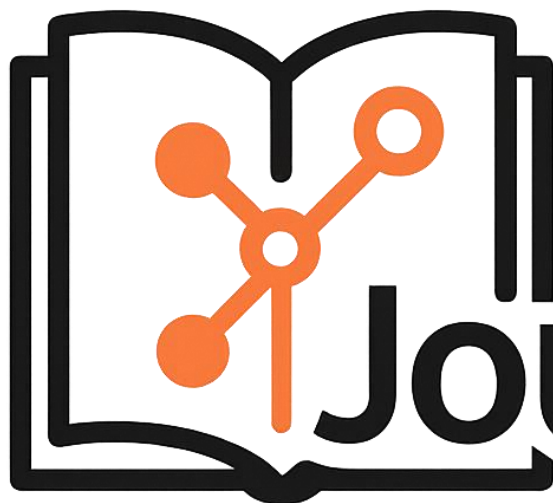




Git Journal



Morgan Dussault
ETML – CIN4A
Chef de projet : Xavier Carrel
Expert 1 : Roger Malherbe
Expert 2 : Michael Wyssa

Table des matières

1	Analyse préliminaire	4
1.1	Introduction	4
1.2	Objectifs.....	4
1.3	Gestion de projet	4
1.4	Planification initiale	5
2	Analyse / Conception.....	6
2.1	Contexte produit	6
2.2	Contextes techniques	7
2.2.1	Opérationnel	7
2.2.2	Validation	7
2.2.3	Développement.....	8
2.3	Concept	9
2.4	Analyse fonctionnelle.....	10
2.4.1	Donner son Token à l'app.....	10
2.4.2	Changer de Repository et d'utilisateur	11
2.4.3	Afficher mes commits sous forme de JDT	13
2.4.4	Reprendre mon JDT sur un autre poste	14
2.4.5	Gérer les entrées	16
2.4.6	Exporter en PDF	22
2.5	Stratégie de test.....	23
2.6	Risques techniques	23
3	Réalisation.....	24
3.1	Gestion des différents fonctionnements.....	24
3.2	Enregistrement et gestion des changements.....	25
3.3	Gestion du PAT	26
3.4	Gestion des Objets	27
3.5	Déroulement	29
3.5.1	Sprints	29
3.5.2	Stories	30
3.6	Description des tests effectués	32
3.7	Code review.....	35
3.8	Bilan.....	35
3.8.1	Erreurs restantes	35
3.8.2	Stories	36
3.8.3	Dette technique.....	37
3.9	Recours à l'intelligence artificielle	38
3.10	Liste des documents fournis	39
4	Conclusions.....	40
4.1	Objectifs atteint et non-atteints	40
4.2	Bilan Personnel.....	40
4.3	Difficultés	40
4.4	Améliorations possibles	41
5	Annexes.....	43

5.1	Résumé du rapport du TPI / version succincte de la documentation	43
5.1.1	Situation de départ.....	43
5.1.2	Mise en œuvre.....	43
5.1.3	Résultat final	43
5.2	Sources – Bibliographie	44
5.3	Journal de travail	44
5.4	Manuel d'Utilisation.....	45
5.4.1	Génération PAT	45
5.5	Glossaire	45

Table des illustrations

1	Schéma Fonctionnement GitJournal	6
2	Matériel Test.....	8
3	Schéma Fonctionnement GitJournal	9
4	Exemple fichiers GitJ	9
5	Maquette Authentification - 1	10
6	Maquette Authentification - 2	10
7	Maquette Authentification - 3	11
8	Maquette Changement repository - 1	12
9	Maquette Changement repository - 2	12
10	Maquette Changement repository - 3	13
11	Maquette Import - 1	14
12	Maquette Import - 2	15
13	Maquette Modification - 1	16
14	Maquette Modification – 2.....	16
15	Maquette Changement Date - 1	17
16	Maquette Changement Date - 2	17
17	Maquette Changement Date - 3	18
18	Maquette Suppression - 1.....	19
19	Maquette Suppression - 2.....	19
20	Maquette Ajout - 1	20
21	Maquette Ajout - 2	20
22	Maquette Ajout - 3	21
23	Maquette Export - 1	22
24	Maquette Export - 2	22
25	Schéma Input/Output GitJournal	24
26	Extrait code List Commit.....	25
27	Extrait code Objet Commit_Info	25
28	Liste fichiers du projet.....	27
29	Extrait code Objet Controller.....	28
30	Exemple Leo IA	38
31	GitJournal Logo	39
32	Icone GitJournal	39
33	Fenêtre guide PAT dans GitJournal	45

1 Analyse préliminaire

1.1 Introduction

Le but de ce projet est de créer une application qui permet aux utilisateurs de générer un journal de travail en utilisant les commit GitHub. Son nom est GitJournal

L'application se base sur la date des commits, la personne ayant effectué le commit et les métadonnées du commit. Les métadonnées comprennent le titre du commit et la description du commit

GitJournal va donc permettre de gagner beaucoup de temps car il permet de ne pas perdre de temps à remplir son journal de travail. De plus il permet d'exporter le JDT en pdf avec une belle mise en page et bien présentable.

Ce projet est très similaire à [ETML-INF/gitjournal](#) qui est fait en Rust. Ma version de GitJournal est une application desktop développée en WPF et .Net 8

1.2 Objectifs

Objectifs de formation :

Ce projet est mon projet de TPI.

Objectifs du produit :

Ce produit doit permettre de créer un JDT en format PDF en se basant sur les commits dans une repository donné.

Il va également permettre à l'utilisateur de modifier les données des commits en supprimer et même en ajouter pour refléter au mieux le travail réalisé.

On peut exporter nos modifications dans un fichier et le réimporter sur un autre poste pour pouvoir transmettre notre rapport au format gitj ou travailler sur plusieurs postes.

1.3 Gestion de projet

Pour ce projet je vais utiliser la méthode agile, elle permet une gestion plus flexible du temps et des tâches et permet grâce au sprint review et user stories validée par le CP de ne pas s'égarer.

Il y aura au total 2 sprints durant ce projet, ils vont durer approximativement 2 semaines.

Lors de ses sprints, il y aura une sprint review avec le chef de projet pour faire un point sur la progression du projet.

Lors du début de chaque journée/ demi-journée je fais un résumé de ce que j'ai fait la fois d'avant et ce que je compte faire aujourd'hui.

Outils :

Pour la planification et le management des user stories, j'utilise GitProject et les issues GitHub.

Pour le versioning, j'utilise GitHub.

Les maquette et schéma sont fait sur Figma et DrawIO.

J'ai utilisé la suite office pour le rapport et mon JDT.

Le JDT est fait sur un canevas de JDT sur Excel crée par l'ETML ce qui simplifie la chose pour moi.

1.4 Planification initiale

No	Objectifs	Durée	Dates	Date Sprint Review	
1	Donner son Token à l'app	~2H	~46H30	05.05 Au 16.05	Vendredi 16.05.2025 9H00
	Changer de Repository	~10H30			
	Afficher mes commits sous forme de JDT	~12H			
	Reprendre mon JDT sur un autre poste	~10H			
	Documentation	~10H			
2	Gérer les entrées	~10H50	~36H50	05.05 Au 16.05	Vendredi 23.05.2025 9H00
	Exporter en PDF	~18H			
	Documentation	~8H			

	28.04 - 02.05 Après Matin Midi	05.05 - 09.05 Après Matin Midi	12.05 - 16.05 Après Matin Midi	19.05 - 23.05 Après Matin Midi	26.05 - 30.05 Après Matin Midi	02.06 - 06.06 Après Matin Midi
Lundi	-	05:35	05:35	05:35	03:10 Examens	03:10 -
Mardi	-	-	-	-	-	-
Mercredi	-	07:15	07:15	07:15	07:15	-
Jeudi	-	- 03:10	- 03:10	- 03:10	Férié	-
Vendredi	07:15	07:15	07:15	07:15	Férié	-

Le total d'heures dans ma planification n'atteint pas les 90H prévue pour ce projet. Dans ces heures, je n'ai pas pris en compte le vendredi 2 mai 2025. C'est le jour où j'ai reçu mon CDC et j'ai pris la décision de ne pas l'inclure dans mon premier sprint. Ce vendredi a duré 7H15.

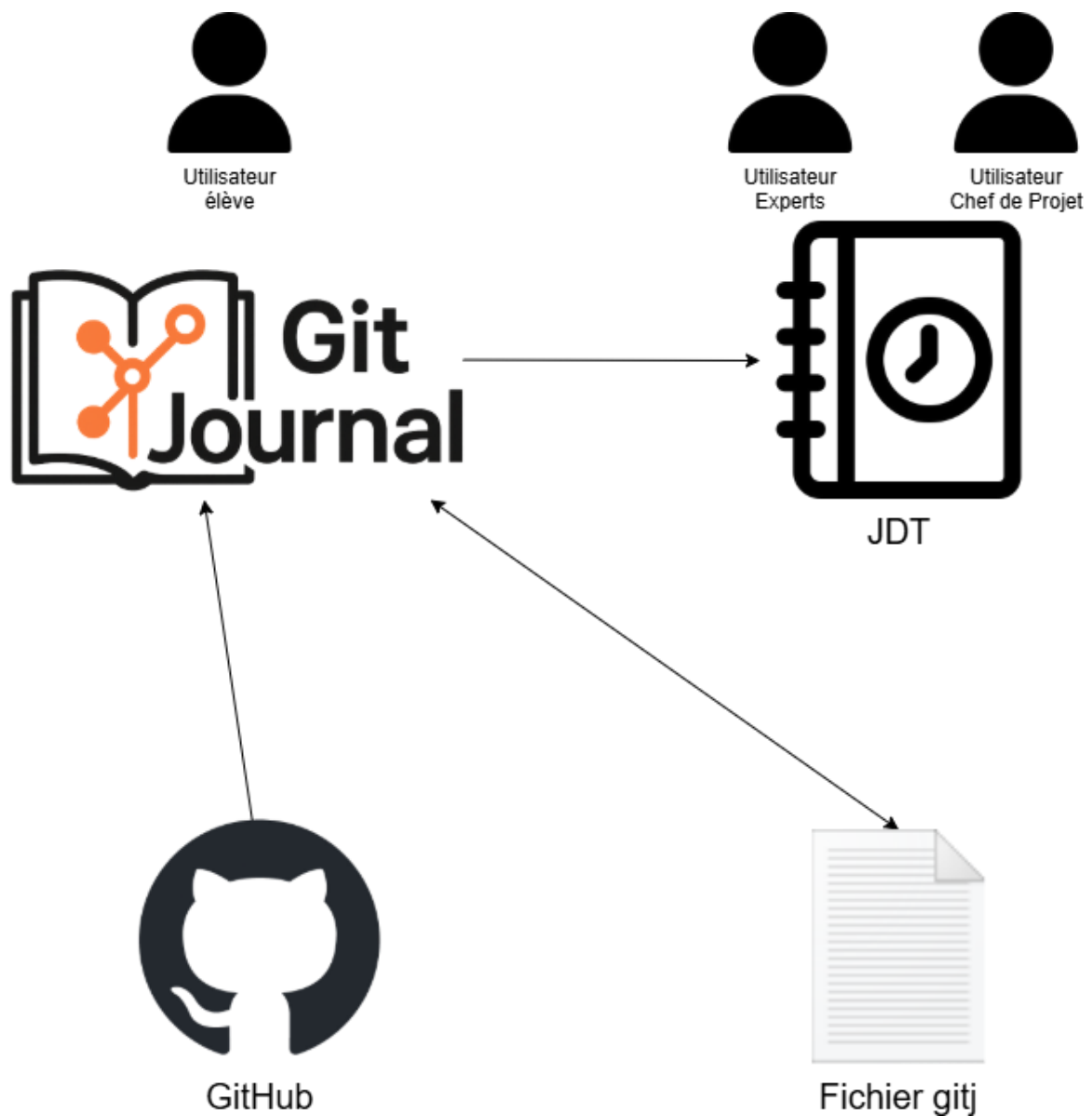
2 Analyse / Conception

2.1 Contexte produit

GitJournal est un program desktop fait pour Windows (10 et 11).

Il est fait pour être utilisé par un seul utilisateur à la fois. Comme l'application requiert un PAT pour se connecter à github, un seul token peut être introduit à la fois ce qui rend l'application mono utilisateur.

Le système exploite les métadata des commits GitHub et prend en compte les modifications de l'utilisateur enregistrées dans un fichier gitj.



1 Schéma Fonctionnement GitJournal

2.2 Contextes techniques

2.2.1 Opérationnel

Pour le contexte opérationnel, GitJournal est exécuté sur n'importe quelle machine sous Windows 10 ou 11.

Avec ou sans droits administrateurs.

Accès à internet et spécialement GitHub, peu importe le lieu, si on veut récupérer des informations depuis GitHub, on va avoir besoin de connexion et que GitHub soit accessible pour afficher des informations de repository ou pour actualiser un fichier gitj. Pour ouvrir un fichier gitj, il faut juste l'application GitJournal.

GitJournal fait appel à l'API de GitHub via PAT (Personal access token).

L'application utilise un installer (exe ou msi) pour installer l'application.

Si on essaye d'ouvrir un fichier. gitj, il s'ouvre automatiquement avec GitJournal

2.2.2 Validation

Les tests de validation se passent sur des PC sous Windows 10 (22H2), excepté ils ne sont pas le poste où se passe le développement.

Un des postes sera mon ordinateur personnel à la maison qui est sous Windows 11.

Les tests sont effectués sur au moins 2 postes.

Il n'y aura pas de droits administrateurs.

Accès à internet et spécialement GitHub.

GitJournal fait appel à l'API de GitHub via PAT (Personal access token)

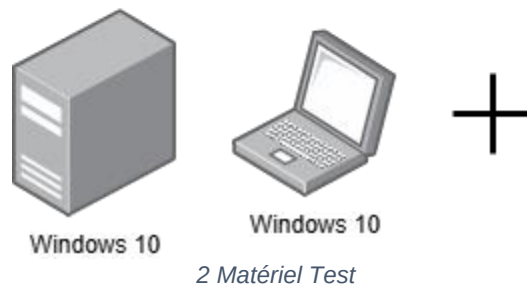
Pour ces tests il faut un repository de quelqu'un qui a comme moi formater son repository de façon que GitJournal soit capable d'utiliser les métadatas pour créer un JDT complet. Cela importe peu à qui appartient le repository, il peut-être le repo de GitJournal. Il faut juste que le PAT soit celui du propriétaire du repository.

Le code est cloné depuis de repository GitHub.

Aucun fichier en plus du code et du fichier de config de l'application (s'il y en a un) ne sont présents.

L'application est lancée depuis un exécutable (sans installeur) généré au préalable sur la machine de dev.

Le programme sera fait en .Net 8, il faudra donc que la machine ait bien .Net 8 d'installé.



2.2.3 Développement

Le développement se passe sur un PC sous Windows 10 (22H2).

Il n'y aura pas de droits administrateurs.

Accès à internet et spécialement à GitHub.

GitJournal fait appel à l'API de GitHub via PAT (Personal access token)

Pour les données, je vais utiliser le repository pour ce projet car j'ai fait en sorte de formater mes commits pour qu'ils soient utilisable par GitJournal.

Liens vers le repository : <https://github.com/Morgan-DD/GitJournal>

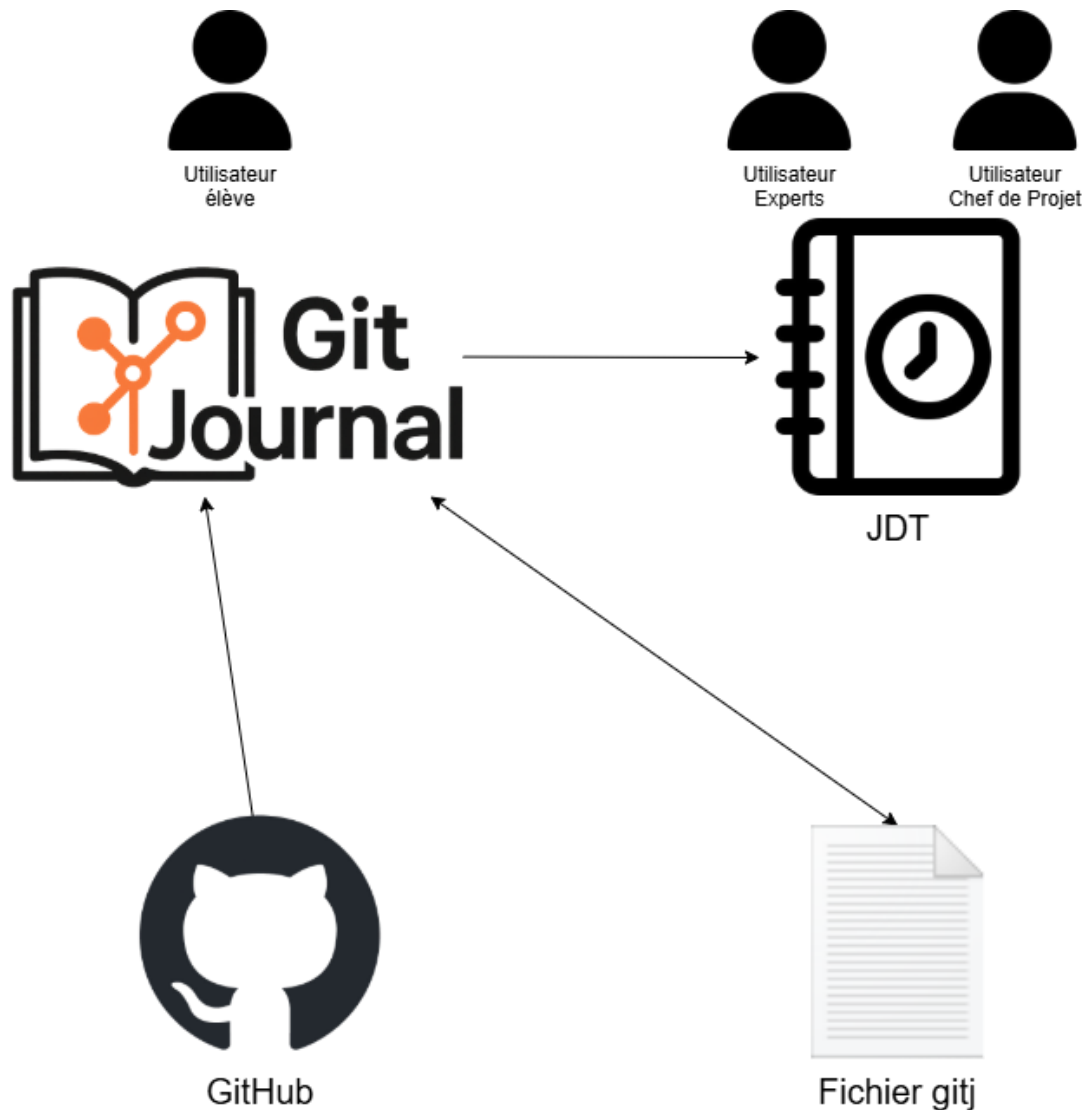
Pour passer de l'environnement de développement à l'environnement de validation, il n'y a pas grand-chose à faire excepté que le repository soit à jour ainsi que compiler le code.

Le développement se passera sur Visual Studio 2022.

Le programme sera fait en .Net 8, il faudra donc que la machine ait bien .Net 8 d'installé.

J'ai choisi .Net 8 car dans les dernières versions de .Net c'est la seule maintenue à long terme (.Net 9 Standard Support Time) et que j'ai déjà utilisé .Net 8 pour mon pré TPI donc je suis familier avec cette version de .Net.

2.3 Concept



3 Schéma Fonctionnement GitJournal

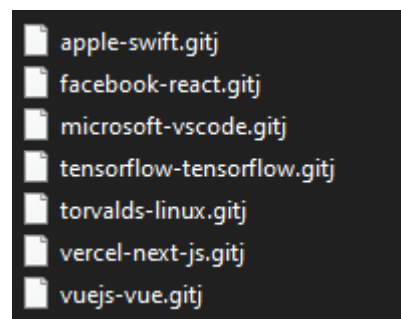
Les données des commits peuvent être récupérées depuis GitHub ou le fichier gitj.

Il y a un fichier gitj par repository, ils sont tous stockés dans un dossier repos par exemple.

Il est possible de partager un fichier gitj et de l'importer comme ça les utilisateurs n'ayant pas accès au repository (si le repo est privé) peuvent aussi lire le JDT directement depuis GitJournal et même le modifier.

En revanche, il ne sera pas mis à jour car étant incapable de synchroniser les informations avec celles de GitHub.

Il est aussi possible d'exporter le JDT en PDF pour une lecture du JDT sans GitJournal installé.



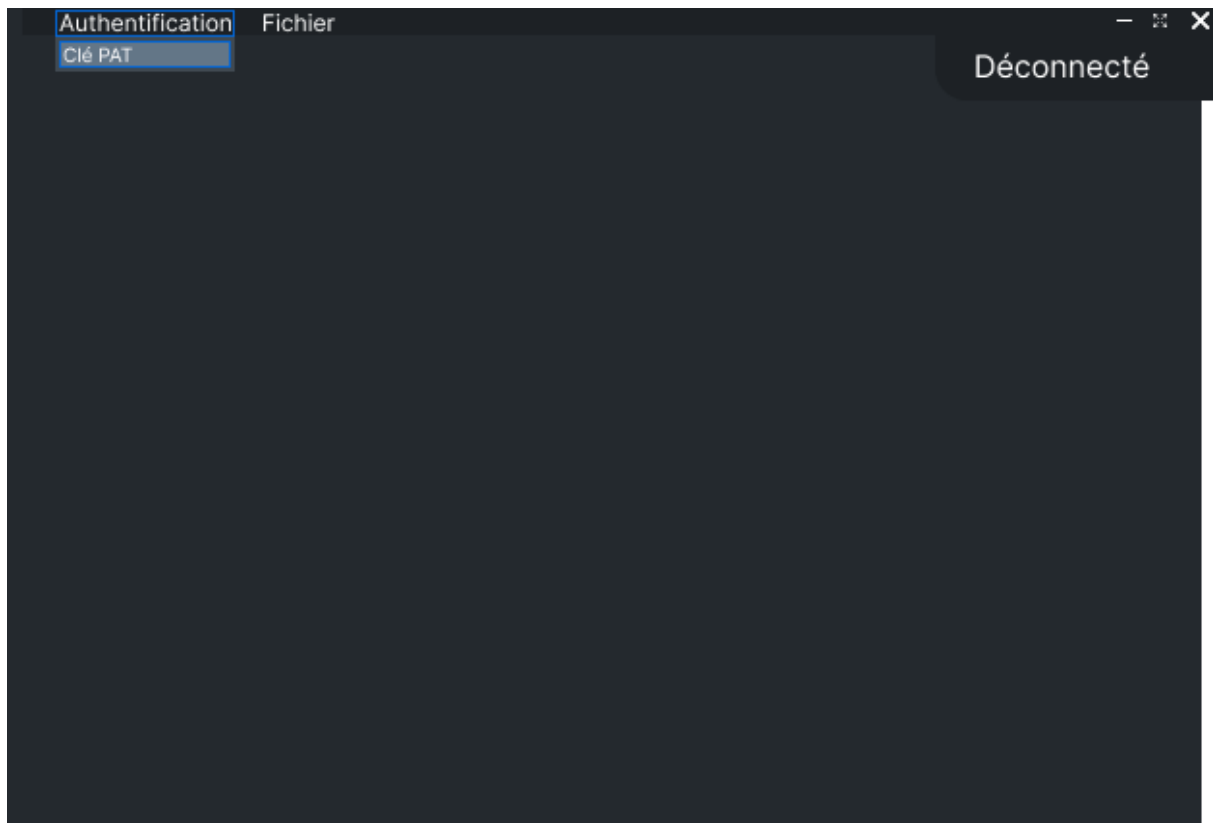
4 Exemple fichiers GitJ

2.4 Analyse fonctionnelle

2.4.1 Donner son Token à l'app

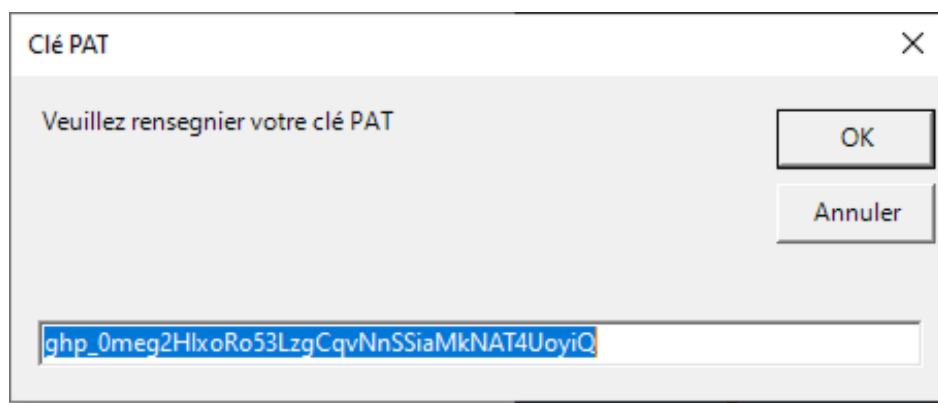
En tant qu'utilisateur, je veux pouvoir m'authentifier en passant par le menu de l'application.

Je clique en haut sur l'option Authentification, puis sur Clé PAT.

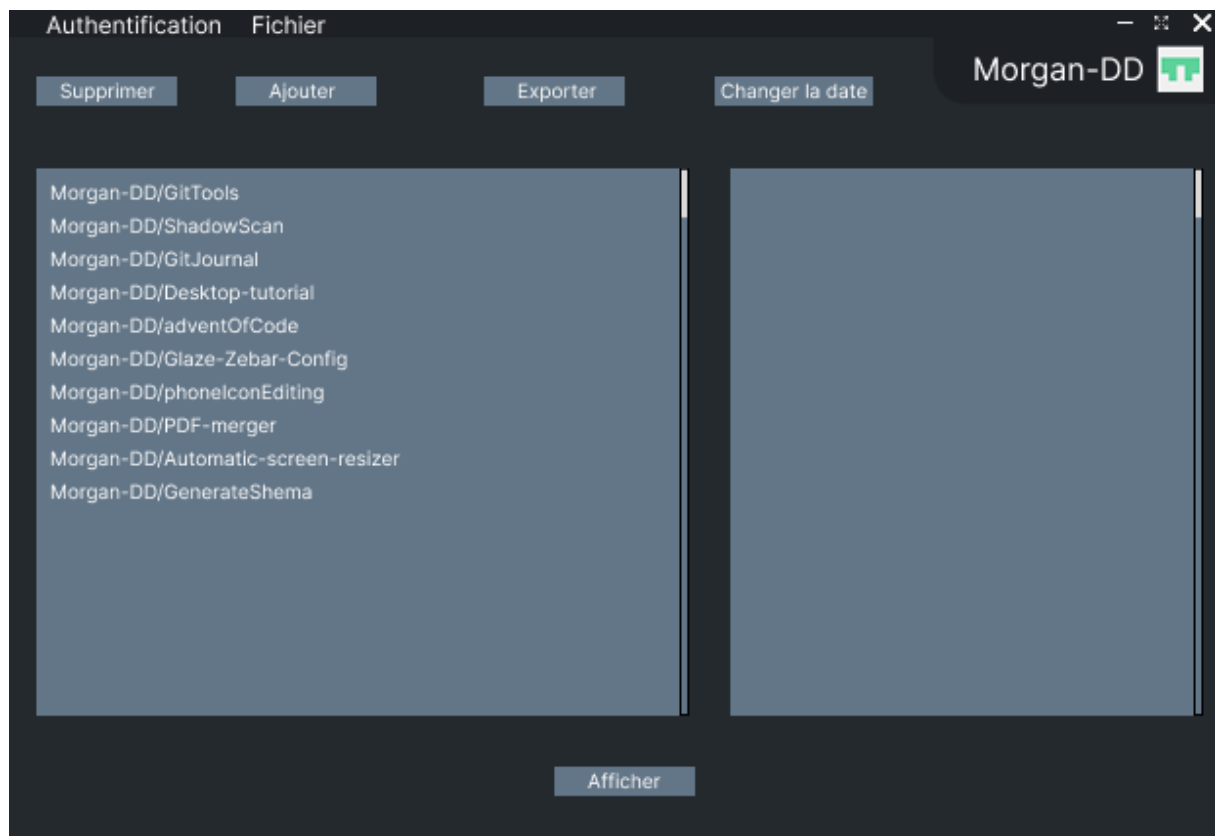


5 Maquette Authentification - 1

Une fenêtre va s'ouvrir et dans celle si, je vais renseigner ma clé PAT.



6 Maquette Authentification - 2



7 Maquette Authentification - 3

Test :

Je lance GitJournal, en haut à droite je vois déconnecté

Dans le menu, j'appuie sur Authentification -> Clé PAT. Une fenêtre apparaît

J'introduis ma clé GitHub, un popup apparaît pour notifier de la réussite de l'action

J'introduis une clé bidon, un popup apparaît pour notifier de l'échec de l'action

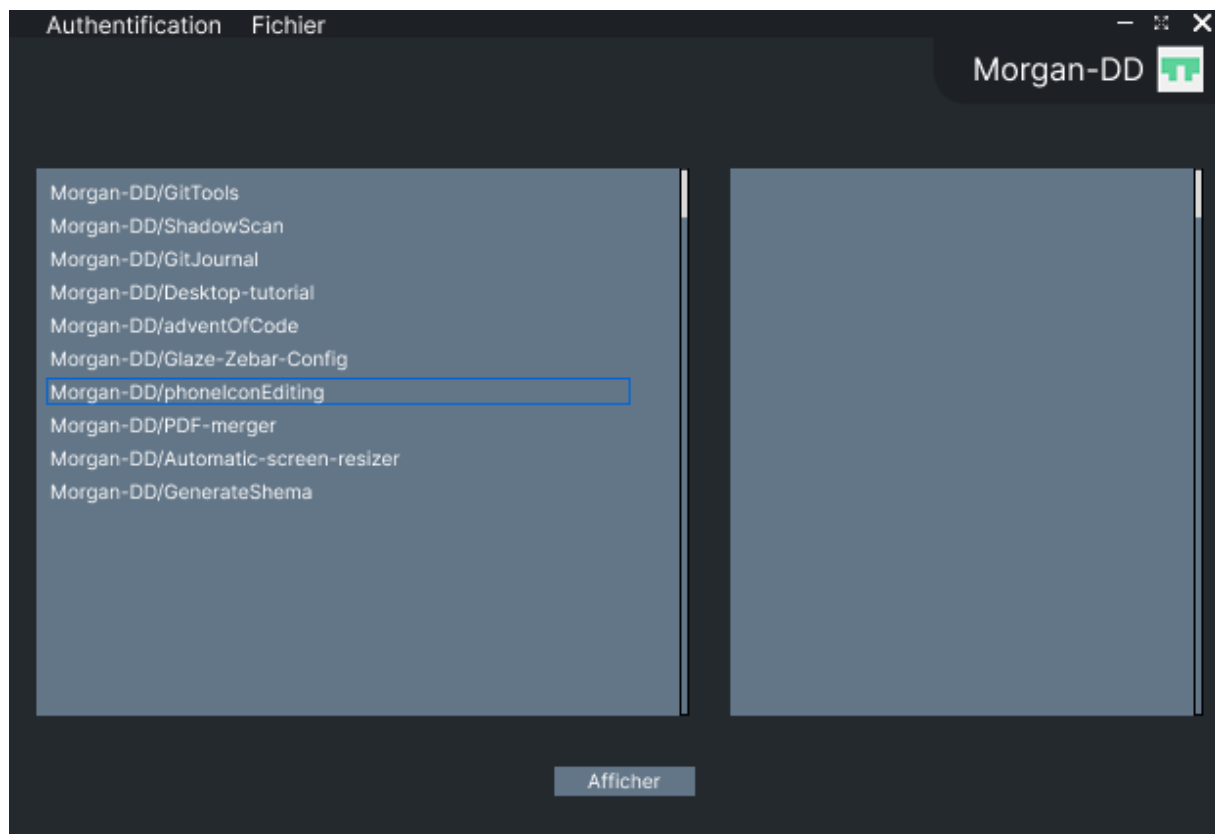
Dans le menu en dessous, un tableau avec la liste de mes repositories apparaît

2.4.2 Changer de Repository et d'utilisateur

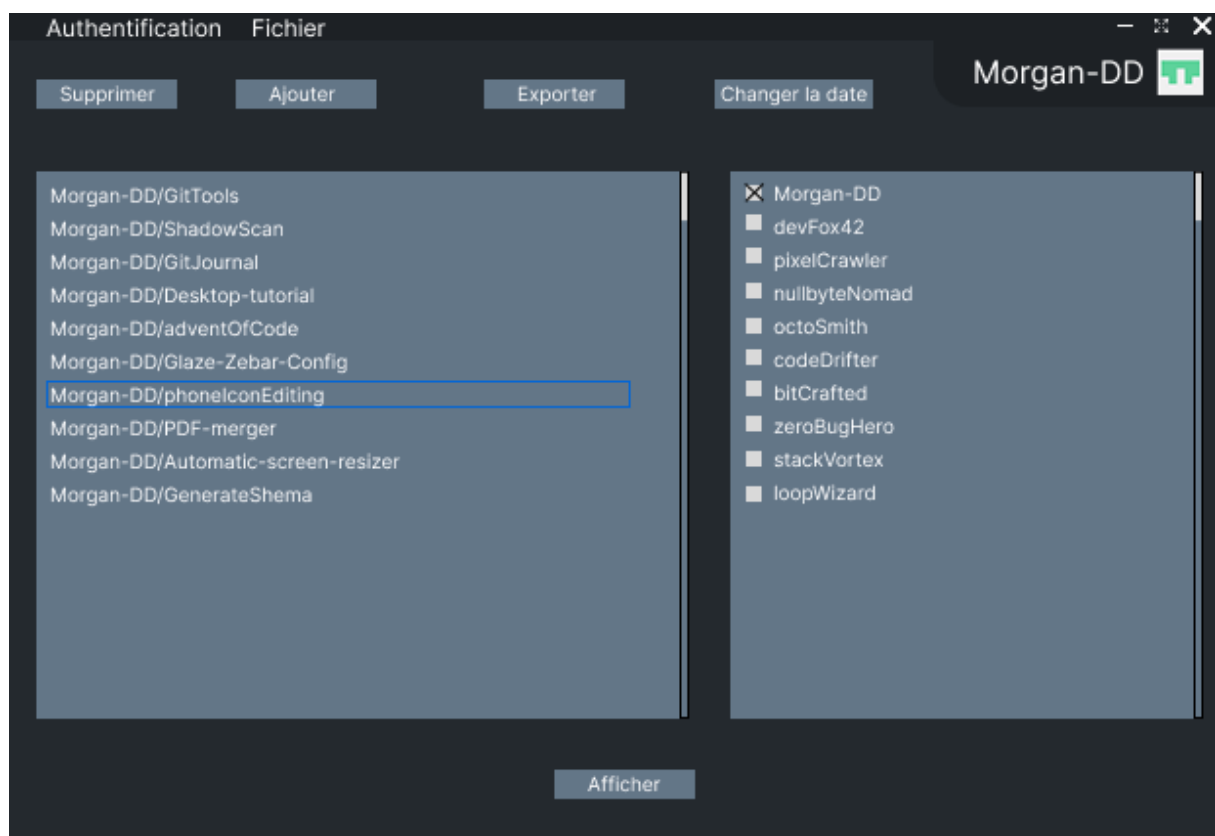
Une fois connecté, en tant qu'utilisateur, je veux pouvoir choisir et changer avec lequel repository GitJournal va générer un JDT.

Dans le menu de gauche, je vois tous mes repository et ceux dans lesquels je suis invité.

Une fois le repository choisi, dans le menu de droite apparaissent les utilisateurs étant invité sur ce repository.



8 Maquette Changement repository - 1



9 Maquette Changement repository - 2

Test :

Je sélectionne un repository dans la liste des repositories. Dans la liste des utilisateurs, toutes les personnes invitées dans le repo apparaissent ainsi que le propriétaire du repo

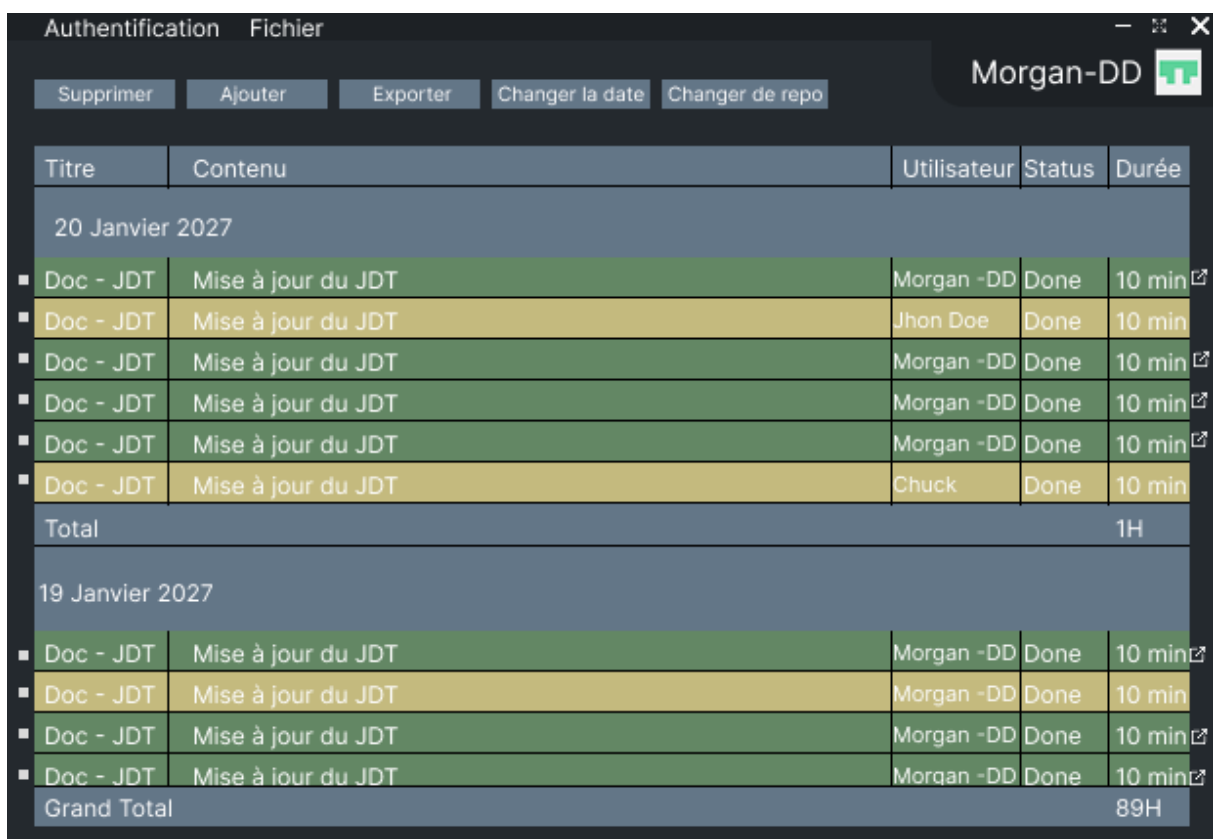
Je coche un ou plusieurs utilisateurs dans la liste

2.4.3 Afficher mes commits sous forme de JDT

En tant qu'utilisateur, je veux pouvoir afficher tous les commits de mon repository sous forme de JDT.

Je veux avoir ces informations pour chaque entrée dans le JDT :

Titre, détails sur l'action effectuée, le nom de l'utilisateur qui l'a réalisé, savoir si c'est toujours en cours ou si c'est une tâche finie et la durée de la tâche.



The screenshot shows a software interface for managing tasks. At the top, there's a header with 'Authentification' and 'Fichier' tabs. Below them are buttons: 'Supprimer', 'Ajouter', 'Exporter', 'Changer la date', and 'Changer de repo'. The user 'Morgan-DD' is logged in. The main area is a table with columns: 'Titre', 'Contenu', 'Utilisateur', 'Status', and 'Durée'. The table is grouped by date, with sections for '20 Janvier 2027' and '19 Janvier 2027'. Each section contains a list of tasks, each with a checkbox, a title, content, user, status, and duration. A 'Total' row is at the bottom of each date group, and a 'Grand Total' row is at the very bottom.

Titre	Contenu	Utilisateur	Status	Durée
20 Janvier 2027				
☐ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min
☐ Doc - JDT	Mise à jour du JDT	Jhon Doe	Done	10 min
☐ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min
☐ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min
☐ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min
☐ Doc - JDT	Mise à jour du JDT	Chuck	Done	10 min
Total				1H
19 Janvier 2027				
☐ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min
☐ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min
☐ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min
☐ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min
Grand Total				89H

10 Maquette Changement repository - 3

Test :

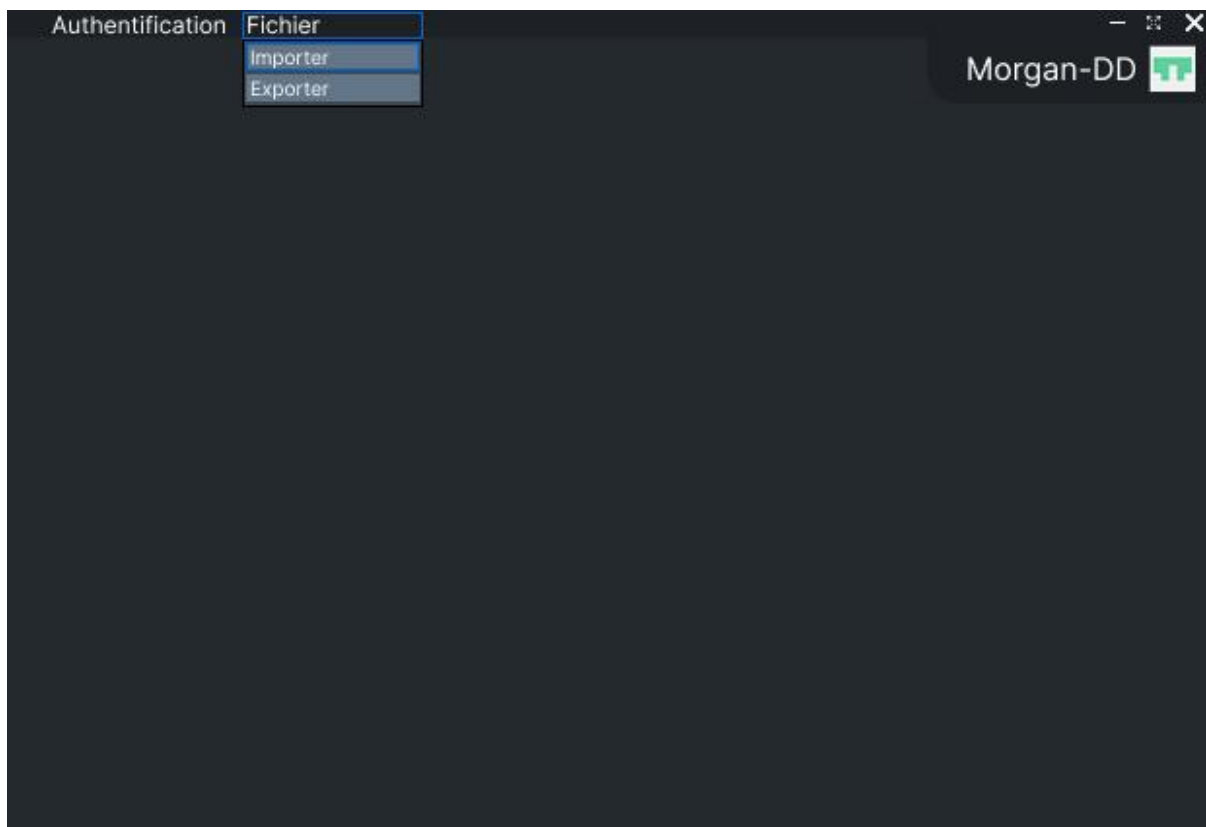
Je sélectionne un repository ainsi que deux utilisateurs puis j'appuie sur Afficher. Il y a plusieurs tableaux qui apparaissent en dessous, un par jour. Je peux défiler dans cette liste de tableaux.

Je choisis un seul utilisateur et la colonne User n'apparaît pas

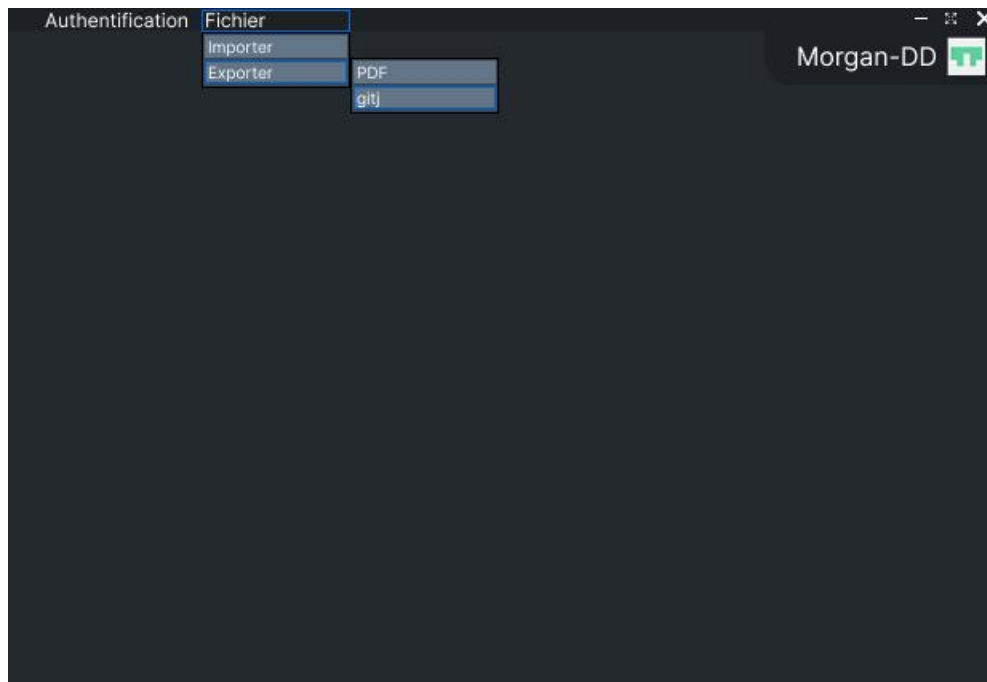
2.4.4 Reprendre mon JDT sur un autre poste

En tant qu'utilisateur, je veux pouvoir faire des modifications sur une story sur un de mes repository. Je change de salle et je peux reprendre mes modifications.

Je veux être capable d'exporter et importer un fichier qui me permet de sauvegarder mes modifications pour un repository donné. Je dois avoir cette option disponible en haut dans le même menu que celui où je donne la clé PAT.



11 Maquette Import - 1



12 Maquette Import - 2

Test :

J'effectue des modifications sur un repository. Je clique sur Fichier, Export, je sauvegarde mon fichier sur une clé USB. Sur cette clé, je retrouve mon fichier gitj.

Je me rends sur un autre poste avec ce fichier que j'ai exporté. Je lance GitJournal, en haut dans le menu je presse sur Fichier puis Importer, je choisis mon fichier gitj puis je retrouve mes modifications.

2.4.5 Gérer les entrées

En tant qu'utilisateur, je veux pouvoir modifier, ajouter et supprimer certaines informations dans le JDT.

Le titre, contenu, statu, temp et la date de l'entrée doivent être modifiables.

En cliquant dans un champ, ça me permet de le modifier et sauvegarde automatiquement.

Authentification Fichier				
Supprimer Ajouter Exporter Changer la date Changer de repo				
Titre		Contenu	Utilisateur	Status
				Durée
20 Janvier 2027				
Doc - JDT			Morgan -DD	Done
Doc - JDT		Mise à jour du JDT	Jhon Doe	Done
Doc - JDT		Mise à jour du JDT	Morgan -DD	Done
Doc - JDT		Mise à jour du JDT	Morgan -DD	Done
Doc - JDT		Mise à jour du JDT	Morgan -DD	Done
Doc - JDT		Mise à jour du JDT	Chuck	Done
Total				1H
19 Janvier 2027				
Doc - JDT		Mise à jour du JDT	Morgan -DD	Done
Doc - JDT		Mise à jour du JDT	Morgan -DD	Done
Doc - JDT		Mise à jour du JDT	Morgan -DD	Done
Doc - JDT		Mise à jour du JDT	Morgan -DD	Done
Grand Total				89H

13 Maquette Modification - 1

Authentification Fichier				
Supprimer Ajouter Exporter Changer la date Changer de repo				
Titre		Contenu	Utilisateur	Status
				Durée
20 Janvier 2027				
Doc - JDT		Remise en forme du JDT (erreurs d'horraire)	Morgan -DD	Done
Doc - JDT		Mise à jour du JDT	Jhon Doe	Done
Doc - JDT		Mise à jour du JDT	Morgan -DD	Done
Doc - JDT		Mise à jour du JDT	Morgan -DD	Done
Doc - JDT		Mise à jour du JDT	Morgan -DD	Done
Doc - JDT		Mise à jour du JDT	Chuck	Done
Total				1H
19 Janvier 2027				
Doc - JDT		Mise à jour du JDT	Morgan -DD	Done
Doc - JDT		Mise à jour du JDT	Morgan -DD	Done
Doc - JDT		Mise à jour du JDT	Morgan -DD	Done
Doc - JDT		Mise à jour du JDT	Morgan -DD	Done
Grand Total				89H

14 Maquette Modification – 2

En cochant la checkbox à gauche des entrées, on peut ensuite cliquer sur [Changer la date] pour pouvoir changer la date de l'entrée, ce qui va la changer de tableau.

Authentification Fichier		Morgan-DD 		
Supprimer Ajouter Exporter Changer la date Changer de repo				
Titre	Contenu	Utilisateur	Status	Durée
20 Janvier 2027				
☐ Doc - JDT	Remise en forme du JDT (erreurs d'horraire)	Morgan -DD	Done	10 min 
☐ Doc - JDT	Mise à jour du JDT	Jhon Doe	Done	10 min
☐ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min 
☐ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min 
☐ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min 
☒ Doc - JDT	Mise à jour du JDT	Chuck	Done	10 min
Total				1H
19 Janvier 2027				
☐ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min 
☐ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min
☐ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min 
☐ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min 
Grand Total				89H

15 Maquette Changement Date - 1

Date

16 Maquette Changement Date - 2

Authentification

Fichier

Supprimer

Ajouter

Exporter

Changer la date

Changer de repo

Morgan-DD



Titre	Contenu	Utilisateur	Status	Durée
20 Janvier 2027				
■ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min 
■ Doc - JDT	Mise à jour du JDT	Jhon Doe	Done	10 min
■ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min 
■ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min 
■ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min 
Total				1H
19 Janvier 2027				
■ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min 
■ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min
■ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min 
■ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min 
■ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min 
Grand Total				89H

17 Maquette Changement Date - 3

Toujours en cochant la checkbox, en appuyant sur le bouton supprimer, cela supprime l'entrée.

Authentification

Fichier

Supprimer

Ajouter

Exporter

Changer la date

Changer de repo

Morgan-DD



Titre	Contenu	Utilisateur	Status	Durée
20 Janvier 2027				
☒ Doc - JDT	Remise en forme du JDT (erreurs d'horraire)	Morgan -DD	Done	10 min 
☐ Doc - JDT	Mise à jour du JDT	Jhon Doe	Done	10 min
☐ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min 
☐ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min 
☐ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min 
☒ Doc - JDT	Mise à jour du JDT	Chuck	Done	10 min
Total				1H
19 Janvier 2027				
☐ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min 
☐ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min
☐ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min 
☐ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min 
Grand Total				89H

18 Maquette Suppression - 1

Authentification Fichier

SupprimerAjouterExporterChanger la dateChanger de repo

Morgan-DD

Titre	Contenu	Utilisateur	Status	Durée
20 Janvier 2027				
Doc - JDT	Remise en forme du JDT (erreurs d'horraire)	Morgan -DD	Done	10 min
Doc - JDT	Mise à jour du JDT	Jhon Doe	Done	10 min
Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min
Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min
Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min
Doc - JDT	Mise à jour du JDT	Chuck	Done	10 min
Total				1H
19 Janvier 2027				
Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min
Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min
Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min
Doc - JDT	Mise à iour du JDT	Morqan -DD	Done	10 min
Grand Total				89H

19 Maquette Suppression - 2

En appuyant sur le bouton [Ajouter] en haut, une fenêtre apparaît nous demandant des informations pour créer une entrée, (toutes sont obligatoires sauf le contenu). Une fois renseignées, cela crée l'entrée à la bonne date.

Authentification Fichier

SupprimerAjouterExporterChanger la dateChanger de repo

Morgan-DD

Titre	Contenu	Utilisateur	Status	Durée
20 Janvier 2027				
■ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min
■ Doc - JDT	Mise à jour du JDT	Jhon Doe	Done	10 min
■ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min
■ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min
■ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min
Total				1H
19 Janvier 2027				
■ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min
■ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min
■ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min
■ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min
■ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min
Grand Total				89H

20 Maquette Ajout - 1

Titre	Meeting
Text	Meting avec chuck noris
Personne	Morgan-DD
Status	☒ DONE ☐ WIP
Durée	15 Min
Date	05.6.2027
Annuler Ajouter	

21 Maquette Ajout - 2

Authentification Fichier		Morgan-DD 		
Supprimer	Ajouter	Exporter	Changer la date	Changer de repo
Titre	Contenu	Utilisateur	Status	Durée
20 Janvier 2027				
☐ Doc - JDT	Remise en forme du JDT (erreurs d'horraire)	Morgan -DD	Done	10 min 
☐ Doc - JDT	Mise à jour du JDT	Jhon Doe	Done	10 min
☐ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min 
☐ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min 
☐ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min 
☐ Doc - JDT	Mise à jour du JDT	Chuck	Done	10 min
☐ Code	Ajout de la fonction pour la sécurité du login	Chuck	Done	3H
Total				4H
19 Janvier 2027				
☐ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min 
☐ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min
☐ Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min 
Grand Total				89H

22 Maquette Ajout - 3

Test :

Je choisi une entrée qui viens depuis GitHub (en vert), je modifie la valeur de cette entrée puis sauvegarde. Le champ devient jaune (signifie la modification).

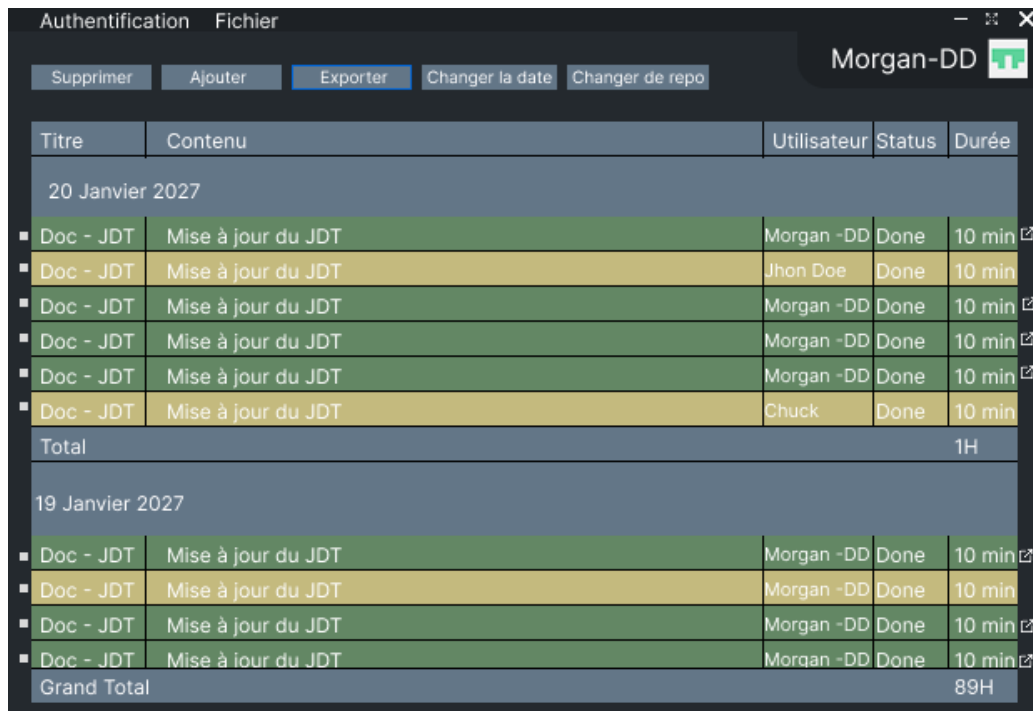
Je choisi une ou plusieurs entrées, je coche la checkbox à droite. Je presse le bouton Supprimer. L'entrée disparaît.

Je choisi une ou plusieurs entrées, je coche la checkbox à droite. Je presse le bouton Modifier. Je choisis une autre date. L'entrée va maintenant apparaitre dans le tableau du jour choisi.

Je presse le bouton Ajouter. Une fenêtre va apparaitre avec plusieurs champs à remplir. Je remplis les champs obligatoires puis presse sur ajouter. Dans le tableau du jour que j'ai choisi apparait ma nouvelle entrée créée à la main. Elle est jaune

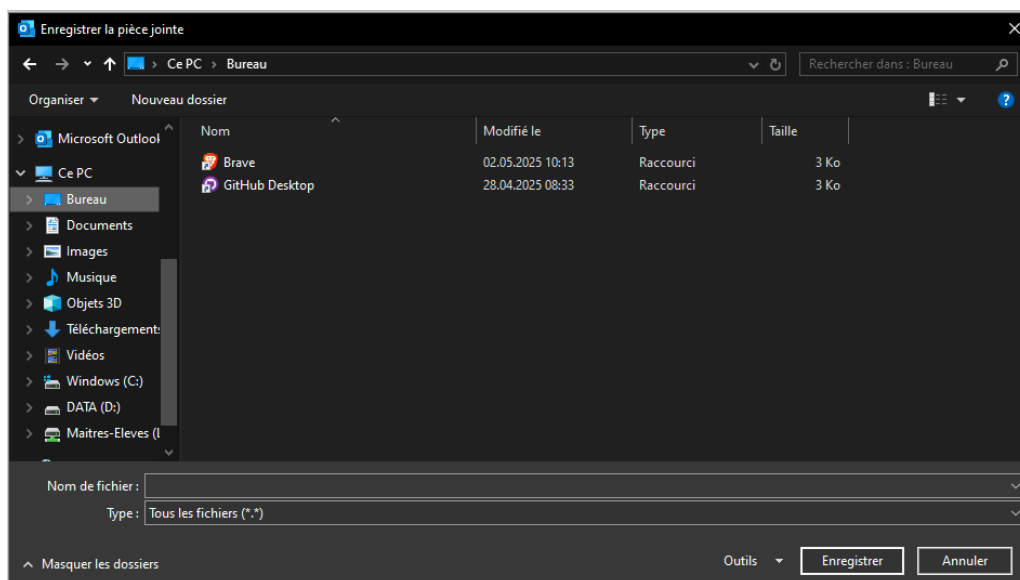
2.4.6 Exporter en PDF

En tant qu'utilisateur, je veux pouvoir exporter mon JDT en PDF. Dans le menu en haut sous fichier, Exporter, je choisis Exporter en PDF. Je choisis le chemin et GitJournal me génère un fichier PDF qui est mon JDT.



Titre	Contenu	Utilisateur	Status	Durée
20 Janvier 2027				
Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min
Doc - JDT	Mise à jour du JDT	Jhon Doe	Done	10 min
Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min
Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min
Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min
Doc - JDT	Mise à jour du JDT	Chuck	Done	10 min
Total				1H
19 Janvier 2027				
Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min
Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min
Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min
Doc - JDT	Mise à jour du JDT	Morgan -DD	Done	10 min
Grand Total				89H

23 Maquette Export - 1



24 Maquette Export - 2

Test :

Dans le menu en haut de page, je presse fichier -> Exporter -> Exporter en PDF, choisis le dossier où exporter le fichier PDF. Dans ce dossier, un PDF est généré.

2.5 Stratégie de test

Les tests seront faits sur plusieurs postes différents pour garantir une certaine sécurité quant au fonctionnement des features testées.

Les tests seront simplement les tests de validation des différentes user stories listée dans le point [Analyse fonctionnelle](#).

Les tests seront faits sans donnée mise en place au préalable sauf si un test d'une des user stories dis le contraire.

Le PAT fait exception à cette règle car sans lui, il est impossible de récupérer les informations depuis GitHub.

Le test peut être effectué par toute personne ayant un repository qui a été formaté pour fonctionner avec GitJournal, c'est-à-dire ajouter dans la description des commits la durée de chaque tâche entre crochets : [1h30], [5min], [3h], ect..

Il faut également que dans la description après l'information de la durée apparaisse le statut [DONE] ou [WIP] (Work In Progress) pour que le commit soit affiché dans GitJournal et dans le fichier PDF généré.

Pour plus de détails sur le contexte de validation, voir section [2.2.2 Validation](#).

2.6 Risques techniques

GitHub :

Comme GitJournal s'appuie sur l'API de GitHub, si l'api ou tout GitHub est down, ça Paralyserait le projet.

Malheureusement, je n'ai pas de moyen de me protéger de ce type de panne car tout mon projet tourne autour de GitHub. Si il me reste une feature qui ne requière pas absolument GitHub comme la génération de PDF, je peux toujours avancer sur une de ces feature mais si cela arrive lors d'une Sprint Review ou quand j'ai absolument besoin, cela va me bloquer.

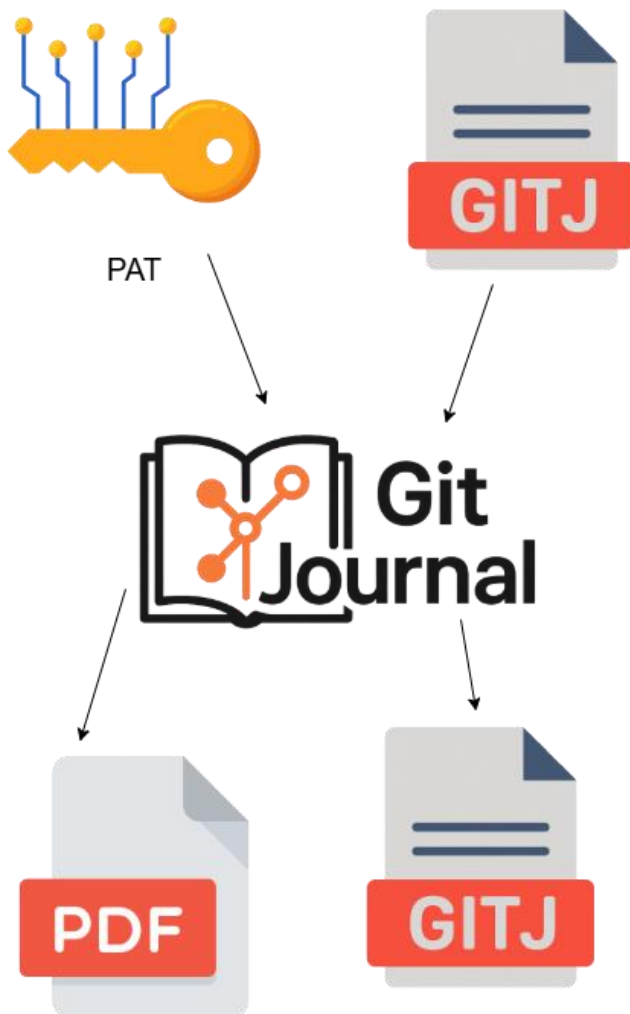
PDF

Je n'ai pas encore utilisé le C# pour manipuler des PDF (partie export du JDT en PDF), cela pourrait prend plus de temp que prévu ou même qui ça me bloque complètement. Pour me protéger, j'ai alloué un temp important pour cette partie du projet qui est pour moi la partie la plus complexe.

3 Réalisation

3.1 Gestion des différents fonctionnements

GitJournal a deux manières de fonctionner, avec le PAT ou un fichier gitj.



25 Schéma Input/Output GitJournal

Pour le PAT :

GitJournal va récupérer tous les repositories dont lesquels l'utilisateur est propriétaire et ceux sur lesquels il est invité également et tout cela une fois s'être connecté avec un PAT valide ayant au minimum les droits sur les repos.

Il peut ensuite choisir un repo ainsi qu'un utilisateur pour générer un JDT.

Si c'est la première fois que l'utilisateur génère un JDT à partir de ce Repo, Le JDT généré est sauvegarder dans un fichier gitj créer automatiquement par le programme.

Au contraire si le fichier existe déjà, GitJournal appond les nouvelles informations (commits) au fichier.

Pour les modifications, il va écrire les modifications dans le fichier gitj qui à ce moment existe à coup sûr.

Pour le fichier gitj :

Quand un utilisateur sans être connecté ouvre un fichier gitj à l'aide de GitJournal, le fichier est simplement affiché, ce qui est plutôt rapide car il ne va pas chercher sur github de potentiels nouveautés.

Quand l'utilisateur effectue des modifications sur ce fichier, les changements seront effectués directement dans le fichier d'origine.

3.2 Enregistrement et gestion des changements

GitJournal reçoit des informations depuis 2 source distinctes, GitHub et du fichier gitj s'il existe.

Les données sont stockées dans un tableau d'un objet custom nommé Commit_Info.

```
public List<Commit_Info> __commits { get; set; }
```

26 Extrait code List Commit

```
public class Commit_Info
{
    11 references
    public string CommitId { get; set; }
    7 references
    public string Title { get; set; }
    8 references
    public string Content { get; set; }
    8 references
    public string User { get; set; }
    6 references
    public string Status { get; set; }
    8 references
    public TimeSpan Duration { get; set; }
    3 references
    public bool ExistingStatus { get; set; }
    7 references
    public DateTime Date { get; set; }
    2 references
    public string Url { get; set; }
    3 references
    public string Origin { get; set; }

    3 references
    public bool IsTitleModified { get; set; }
    3 references
    public bool IsContentModified { get; set; }
    2 references
    public bool IsUserModified { get; set; }
    3 references
    public bool IsStatusModified { get; set; }
    3 references
    public bool IsTDurationModified { get; set; }
}
```

27 Extrait code Objet Commit_Info

En premier GitJournal vérifie si un fichier gitj pour le repo actuel existe, si oui il récupère les données venant de ce fichier et les ajoute à ce tableau.

Après cette vérification, que le fichier existe ou pas, GitJournal va chercher les commit dans GitHub et les ajoute au tableau.

S'il y a un commit à double (CommitId fonctionne comme clé primaire) alors GitJournal priorise les commits venant du fichier gitj. Les deux types de commits sont différenciable à l'aide de la valeur de Origin "GitJ" ou "GitHub". Donc les commits dont l'origine est "GitJ" sont préférés à ceux dont l'origine est "GitHub".

Si les deux commits en conflits ont la même origine, GitJournal priorise le plus récent des deux.

Une fois cette étape passée, GitJournal va enregistrer dans le fichier gitj le tableau _commits sous forme de json. Il va changer l'origine de tous les commits qu'il enregistre en "GitJ".

3.3 Gestion du PAT

Lors de la première utilisation de GitJournal, l'utilisateur est déconnecté. Il faut alors que l'utilisateur donne son PAT à GitJournal.

Guide pour la génération du PAT : [5.5.1 Génération PAT](#).

Lors de première connexion, le PAT est stocké et l'app démarre normalement. Lors de la prochaine utilisation sur le même poste, GitJournal va récupérer automatiquement le PAT sauvegardé et connecter l'utilisateur.

Une fois connecté, que cela soit manuel ou automatique, GitJournal va tester le PAT. S'il est invalide, un popup va en informer l'utilisateur, si au contraire, il est valide alors GitJournal va récupérer le nom de l'utilisateur ayant généré le PAT ainsi que son image de profil et afficher cela en haut à droite dans l'app.

Ensuite, il récupère les noms de repositories auquel le PAT lui donne accès.

3.4 Gestion des Objets

Pour cet app j'ai utilisé beaucoup d'user control et d'objets.

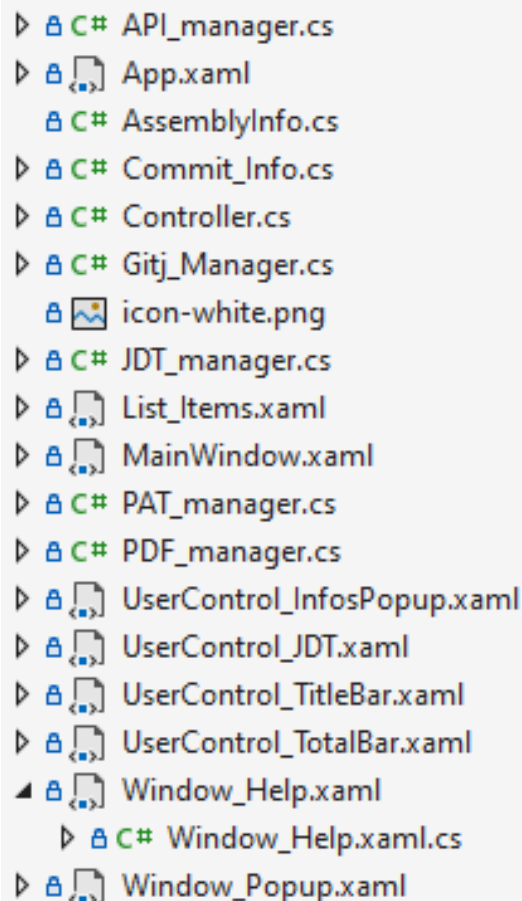
Pour gérer tous ces objets et pour simplifier la communication, je me suis grandement inspiré du [MVC](#) (Model View Controller).

J'ai un objet que j'ai appelé Controller qui permet de relier tous les objets entre eux, ils sont référencés dans Controller. Il a aussi certaines variables utilisées par plusieurs objets donc elles sont disponibles pour tous.

Cela permet la communication entre chaque objets et l'appel de méthode d'autres objets plus facilement.

Tous objet ayant besoin de contacter un autre objet ou d'accéder aux variables du Controller ont une référence vers celui-ci.

Presque tous les objets sont instanciés dans le Controller et le Controller, qui leur transmet une référence à lui-même lors de leur initialisation.



- API_manager.cs
- App.xaml
- AssemblyInfo.cs
- Commit_Info.cs
- Controller.cs
- Gitj_Manager.cs
- icon-white.png
- JDT_manager.cs
- List_Items.xaml
- MainWindow.xaml
- PAT_manager.cs
- PDF_manager.cs
- UserControl_InfosPopup.xaml
- UserControl_JDT.xaml
- UserControl_TitleBar.xaml
- UserControl_TotalBar.xaml
- Window_Help.xaml
 - Window_Help.xaml.cs
- Window_Popup.xaml

28 Liste fichiers du projet

Aperçu du Controller :

```
public class Controller
{
    10 references
    public MainWindow _mainWindow { get; set; }
    3 references
    public PAT_manager _PATmanager { get; set; }
    4 references
    public API_manager _APImanager { get; set; }
    2 references
    public Gitj_Manager _Gitjmanager { get; set; }
    24 references
    public JDT_manager _JDTmanager { get; set; }

    2 references
    public PDF_manager _PDFmanager { get; set; }

    3 references
    public List_Items _RepoList { get; set; }
    4 references
    public List_Items _UserList { get; set; }
    4 references
    public UserControl_TitleBar _TitleBar { get; set; }
    3 references
    public UserControl_TotalBar _TotalBar { get; set; }
    8 references
    public UserControl_JDT _JDT { get; set; }
    7 references
    public UserControl_InfosPopup _InfosPopup { get; set; }

    6 references
    public Window_Popup _Window_Popup { get; set; }

    5 references
    public string _PATToken { get; set; }
    6 references
    public string _ClientName { get; set; }
    3 references
    public BitmapImage _ClientAvatar { get; set; }

    7 references
    public string _RepoSelected { get; set; }

    public string _GitJFileDir = "GitJ_Files";

    public bool _isFromGitHub = false;
    public string _ActualGitJPath = "";
    7 references
    public SolidColorBrush _Yellow { get; set; } = (SolidColorBrush)(new BrushCo
    0 references
    public SolidColorBrush _Green { get; set; } = (SolidColorBrush)(new BrushCo
    2 references
    public string _TokenBase { get; set; } = "Created_Token";
```

29 Extrait code Objet Controller

3.5 Déroulement

3.5.1 Sprints

Sprint 1

User stories planifiée :

[Donner son Token à l'app](#)

[Reprendre mon JDT sur un autre poste](#)

[Changer de Repository et d'utilisateur](#)

[Afficher mes commits sous forme de JDT](#)

Voir [2.4 Analyse fonctionnelle](#) pour plus d'informations.

De ces 4 stories, seulement 2 ont été validée :

[Donner son Token à l'app](#)

[Reprendre mon JDT sur un autre poste](#)

Les 2 autre ont eu de petit bug :

Pour " [Changer de Repository et d'utilisateur](#) " :

Le problème est que pas tous les repositories sont affichés dans la liste.

Le CP a suggéré que cela pourrait être dû à de la pagination du côté de l'API de GitHub.

Pour " [Afficher mes commits sous forme de JDT](#) " :

Pour cette story, tous les tests sont passé excepté 1. S'il y a du Markdown dans le titre ou contenu du commit, il n'est pas mis en page proprement mais simplement affiché comme du texte brut.

Les tests de validation ont été effectués sur une machine de l'école qui n'est pas celle où le développement c'est passé. Ils ont également été fait sur ma machine personnelle prise à distance en RDP.

Cela a permis d'avoir un plus large échantillon de machines pour les tests.

J'ai bien avancé sur ce sprint, j'ai été bloqué sur la gestion et affichage du Markdown ce qui m'a fait perdre du temps mais à final je ne suis pas en retard.

Pour le sprint 2, je vais commencer par fixer les bugs découverts durant la Sprint Review. Puis suivre la planification initiale.

Sprint 2

User stories planifiée :

[Gérer les entrées](#)

[Exporter en PDF](#)

User stories ayant des bugs lors de la dernière Sprint Review :

Changer de Repository et d'utilisateur
Afficher mes commits sous forme de JDT

Voir [2.4 Analyse fonctionnelle](#) pour plus d'informations.

De ces 4 stories, seulement 3 ont été validée :

[Gérer les entrées](#)

[Exporter en PDF](#)

[Changer de Repository et d'utilisateur](#)

La story " [Afficher mes commits sous forme de JDT](#) " n'a pas été validée dû à une feature non faite, je n'ai pas implémenté l'affichage du Markdown.

Ce sprint a été très productif et j'ai réussi à finir quasiment toutes les stories ainsi que corriger les stories n'ayant pas été validées lors de la SR 1.

Cette sprint review ne se trouve pas à la fin du sprint 2 mais une semaine avant sa fin car le nombre d'heures à ma disposition la semaine prochaine est grandement réduit. Durant les ~13H de la semaine prochaine je compte finir ma doc et si j'ai dû temp en plus repasser sur la story non complétée et sur les améliorations rapides. Voir [3.7.3 Dette technique](#) et [3.7.1 Erreurs restantes](#) pour plus d'informations sur ces petits problèmes et features manquantes ou avec des aspects insuffisants.

3.5.2 Stories

[Donner son Token à l'app :](#)

Pour cette story, j'ai réutilisé la technique que j'avais utilisé dans mon pré TPI cela m'a donc facilité la vie. Je connaissais déjà les mécanismes et erreurs potentielles ce qui m'a permis de réaliser cette story rapidement et sans perturbations.

Elle a été validée lors de la SR 1.

[Reprendre mon JDT sur un autre poste :](#)

Pour cette story, j'avais pensé et designer la structure du fichier gitj qui est enfaite un fichier json. J'ai donc designer un objet qui match la structure de mon fichier pour la manipulation des données du fichier.

Cela m'a pris un peu de temp à mettre en place au début mais une fois fait, cela m'a fait gagner beaucoup de temp et pas que sur cette story.

J'ai réalisé cette story avant la story qui sauvegarde les données récupérées depuis GitHub donc je me suis servi de l'IA (ChatGPT) pour générer des données bidon pour mes tests.

Cette story m'a pris le temp que j'avais planifié (~10H) et ne m'a pas posé de problèmes

Elle a été validée lors de la SR 1.

[Changer de Repository et d'utilisateur :](#)

Cette story devait être réalisée lors du sprint 1, ce qui a été fait mais ayant des bugs elle a été validée lors de la sprint review 2.

Lors du sprint 1, tout s'est bien passé jusqu'à la sprint review. Je n'avais pas remarqué mais pas tous mes repositories s'affichaient. Cela était dû à de la pagination comme le CP l'avait suggéré.

Juste après la sprint review 1 ou se problème a été découvert, je me suis mis à fond dessus et l'ai fixé plutôt facilement.

Elle a finalement été validée lors de la SR 2.

[Afficher mes commits sous forme de JDT :](#)

De toutes les User stories, celle-ci est la seule qui n'a pas été validée.

À la base, cette story était planifiée pour le sprint 1.

Je l'ai réalisée durant le sprint 1, mais sans le markdown car à ce moment j'avais choisi de ne pas le réaliser pour continuer.

Lors de la sprint Review 1 tout était ok sauf le markdown.
Pour la Sprint Review 2, même résultat.

[Gérer les entrées :](#)

Cette story n'a pas été difficile ou simple, elle s'est déroulée plutôt simplement. Durant cette story j'ai dû modifier la structure de mon fichier gitj(json) et donc aussi la structure de l'objet qui me sert à stocker ces infos.

J'ai rajouté des variables pour savoir quels champs ont été modifiés ce qui me sert à l'affichage (jaune modifié, vert de base).

Elle a été validée lors de la SR 2.

[Exporter en PDF :](#)

Pour cette story, j'ai changé de librairie en cours de route.
Au départ, j'avais choisi itext7 comme librairie pour la gestion du fichier PDF.
Après avoir tester des choses avec je me suis rendu compte que ça allait être trop compliqué de faire ce que je voulais avec. J'ai bifurqué sur PdfSharp qui me paraissait plus simple et effectivement car c'est la librairie que j'utilise.

Hormis ça, je n'ai pas eu de problème sur ce point. Cependant je n'avais jamais fait ça avant ce qui a rendu la tâche plus complexe.

Elle a été validée lors de la SR 2.

3.6 Description des tests effectués

Ici vont être listés tous les tests que j'ai effectué lors de ce projet. Ils vont être séparés par user story car elles ont leur propre test.

Ces tests ont été effectués lors des 2 sprint review avec le chef de projet.

[Donner son Token à l'app :](#)

Validé	Je lance GitJournal, en haut à droite je vois déconnecté
Validé	Dans le menu, j'appuie sur Authentification -> Clé PAT. Une fenêtre apparaît
Validé	J'introduis ma clé GitHub, un popup apparaît pour notifier de la réussite de l'action
Validé	J'introduis une clé bidon, un popup apparaît pour notifier de l'échec de l'action
Validé	Dans le menu en dessous, un tableau avec la liste de mes repositories apparaît

Tous les tests sont passés du premier coup sans problèmes.
Ils ont été validés lors de la Sprint Review 1.

[Reprendre mon JDT sur un autre poste :](#)

Validé	J'effectue des modifications sur un repository. Je clique sur Fichier, Export, je sauvegarde mon fichier sur une clé USB. Sur cette clé, je retrouve mon fichier gitj.
Validé	Je me rends sur un autre poste avec ce fichier que j'ai exporté. Je lance GitTools, en haut dans le menu je presse sur Fichier puis Importer, je choisis mon fichier gitj puis je retrouve mes modifications.

Tous les tests sont passés du premier coup sans problèmes.
Ils ont été validés lors de la Sprint Review 1.

Changer de Repository et d'utilisateur :

Validé	Je sélectionne un repository dans la liste des repositories. Dans la liste des utilisateurs, toutes les personnes invitées dans le repo apparaissent ainsi que le propriétaire du repo
Validé	Je coche un ou plusieurs utilisateurs dans la liste

Lors des premiers tests durant la Sprint Review 1, pas tous les repositories s'affichaient. Le problème était de la pagination du côté de l'API de GitHub. Le problème a été fixé lors du sprint 2.

Ils ont été validés lors de la Sprint Review 2.

Gérer les entrées :

Validé	Je choisis une entrée qui vient depuis GitHub (en vert), je modifie la valeur de cette entrée puis sauvegarde. Le champ devient jaune (signifie la modification).
Validé	Quand je modifie un champ, je clique sur un autre et le champ que je modifiais est sauvegardé.
Validé	Quand je modifie un champ, je clique sur un Enter et le champ que je modifiais est sauvegardé.
Non Validé	Quand je modifie un champ, je presse sur un TAB, le champ que je modifiais est sauvegardé et je passe au champ d'après sur la même ligne et si je suis sur la dernière colonne, je passe à la ligne d'après.
Validé	Je modifie un champ, je presse Ctrl+Enter, je passe à la ligne.
Non Validé	Je modifie un champ, j'ajoute du Markdown, quand je valide les modifications, le Markdown est mis en page proprement
Validé	Je choisis une ou plusieurs entrées venant de GitHub, je coche la checkbox à droite. Je presse le bouton Supprimer. L'entrée disparaît.
Validé	Je choisis une ou plusieurs entrées créées à la main, je coche la checkbox à droite. Je presse le bouton Supprimer. L'entrée disparaît.
Validé	Je choisis une ou plusieurs entrées, je coche la checkbox à droite. Je presse le bouton Modifier. Je choisis une autre date. L'entrée va maintenant apparaître dans le tableau du jour choisi.
Validé	Je presse le bouton Ajouter. Une fenêtre va apparaître avec plusieurs champs à remplir. Je remplis les champs obligatoires puis presse sur ajouter. Dans le tableau du jour que j'ai choisi apparaît ma nouvelle entrée créée à la main. Elle est jaune

2 tests ne sont pas passés ; l'affichage du Markdown et le déplacement entre les champs en utilisant TAB et shift+TAB.

Les tests qui ont été passés ont été effectués durant la Sprint Review 2.

[Afficher mes commits sous forme de JDT :](#)

Validé	Je sélectionne un repository ainsi que deux utilisateurs puis j'appuie sur Afficher. Il y a plusieurs tableaux qui apparaissent en dessous, un par jour. Je peux défiler dans cette liste de tableaux.
Validé	Je choisis un seul utilisateur et la colonne User n'apparaît pas
Validé	Quand je scrolle dans le JDT, le haut affichant les titres de chaque colonne et le bas contenant le grand total restent fixes, seulement le contenu bouge
Validé	Je crée une entrée, tout à droite de celle-ci, rien n'apparaît. Sur les entrées venant de GitHub et celles qui sont modifiées, on voit un petit logo qui quand cliqué, nous redirige sur la page GitHub de ce commit.
Non Validé	Les commits contenant plusieurs lignes ou du Markdown sont bien affichés.

Tous les tests sont bons sauf l'affichage du Markdown. Il est affiché comme du simple texte. C'est une fonctionnalité importante ce qui fait que cette story est la seule non validée.

Les tests qui ont été passés ont été effectués durant la Sprint Review 2.

La story est la seule à ne pas avoir été validée.

[Exporter en PDF :](#)

Validé	Dans le menu en haut de page, je presse fichier -> Exporter -> Exporter en PDF, choisis le dossier où exporter le fichier PDF. Dans ce dossier, un PDF est généré.
Validé	Quand je double clique sur le PDF, il s'ouvre avec le lecteur de PDF par défaut
Validé	Le PDF contient un entête avec le nom du projet à l'intérieure
Validé	Les marges du fichier sont propres et les tableaux ne sont pas collés au bord
Validé	Le contenu multiligne est affiché proprement
Non Validé	Il n'y a pas de fraction de tableau ou de date sur une page tout seul
Non Validé	Il y a le total d'heures affiché pour chaque entrée et le grand total
Validé	Le fichier contient les mêmes informations que le JDT affiché dans GitJournal

2 tests n'ont pas été validés. Il y a parfois des bouts de tableau tout seul sur une page et puis le reste sur un autre.

Le grand total d'heures du projet n'est pas affiché.

Les tests qui ont été passés ont été effectués durant la Sprint Review 2.

3.7 Code review

La code review a eu lieu le 26.05.2025, je l'ai fait avec mon CP.

Lors cette code review, 2 problèmes ont été identifiés :

Le manque de commentaire dans le code ainsi que des parties de code commentée pour le debug et laissée tel quel.

La deuxième est le fait que je n'ai pas suivi les normes de codage de l'ETML.

3.8 Bilan

3.8.1 Erreurs restantes

Lors des tests, j'ai trouvé seulement une erreur.

Dans le PDF, j'ai fait en sorte que les longues descriptions soient "wrap" c'est-à-dire que des retours sont ajoutés pour éviter que le texte dépasse sur d'autre cases.

En revanche les très longs mots ne sont pas pris en compte ce qui certaine fois cause des petits bugs visuels.

Lors des tests, c'est arrivée avec un long lien qui était dans la description.

C'est un problème rencontré que 1 seule fois lors des tests ce qui me fait penser que ce problème est rare.

En revanche il peut rendre l'entrée illisible. S'il déborde sur la case d'après Utilisateur/Statuts/Durée, les différents textes se superpose ce que rend la fin de la ligne surchargée.

Pour y remédier, je vois 2 possibilités.

L'une serait de couper le lien et de faire un retour à la ligne. Cette option pourrait donner des blocks de liens ce qui n'est pas super lisible ce qui était le problème à la base.

La seconde option serait de transformer le lien en hyperlien avec le nom du site/titre de la page pour texte. Cela permettrait de raccourcir les liens, éviter les liens sur plusieurs lignes et de solutionner le problème initial.

3.8.2 Stories

[Donner son Token à l'app :](#)

Tout a été réalisé dans cette story.

[Reprendre mon JDT sur un autre poste :](#)

Tout a été réalisé dans cette story.

[Changer de Repository et d'utilisateur :](#)

Tout a été réalisé dans cette story.

[Afficher mes commits sous forme de JDT :](#)

Le Markdown n'est pas interprété mais affiché comme du simple texte.
Le reste a été effectué comme planifié.

[Gérer les entrées :](#)

Après avoir cliqué sur un champ pour le modifier, on ne peut pas naviguer entre les champs en utilisant TAB et Shift+Tab.
Le reste a été effectué comme planifié.

[Exporter en PDF :](#)

3 choses posent problèmes pour cette story :

Comme pour l'affichage dans GitJournal, le Markdown n'est pas interprété mais affiché comme du simple texte.

Parfois, il y a des parties de tableau orpheline. C'est-à-dire que juste la ligne avec le titre des colonnes ou du total d'heures est tout seul sur une page séparé du reste de son groupe.

Le grand total d'heure n'apparaît pas du tout dans le PDF.

Le reste a été effectué comme planifié.

3.8.3 Dette technique

Gestion du PAT :

Cela reprend le point spécifique [3.3 Gestion du PAT](#).

Dans la version actuelle de GitJournal, le PAT est stocké en claire dans un fichier généré par GitJournal.

Quand l'utilisateur s'est connecté une fois, le PAT est sauvegardé dans un fichier nommé GitJournal.Debug.

ATTENTION !! Le PAT est écrit en claire dans ce fichier ce qui n'est pas du tout une bonne pratique. J'ai fait ça de cette manière car trouver et mettre en place un moyen vraiment efficace pour sécuriser ce PAT est une tâche longue et complexe. Malheureusement je n'ai pas le temps de mettre en place cette bonne pratique car le temps me fait défaut. J'avais planifié dès le début que le stockage du PAT serait lacunaire.

Affichage du JDT dans GitJournal et dans le PDF :

Cela reprend le point spécifique [2.4.3 Afficher mes commits sous forme de JDT](#) et [2.4.6 Exporter en PDF](#).

Dans GitJournal et dans le PDF, le Markdown n'est pas interprété mais simplement écrit tel-quel.

La résolution de ce problème n'est pas trop complexe, il suffit par exemple de convertir le Markdown en HTML puis de l'afficher dans une WebView2 par exemple. On peut y rajouter des paramètres pour modifier l'affichage pour qu'il se fonde dans le décor.

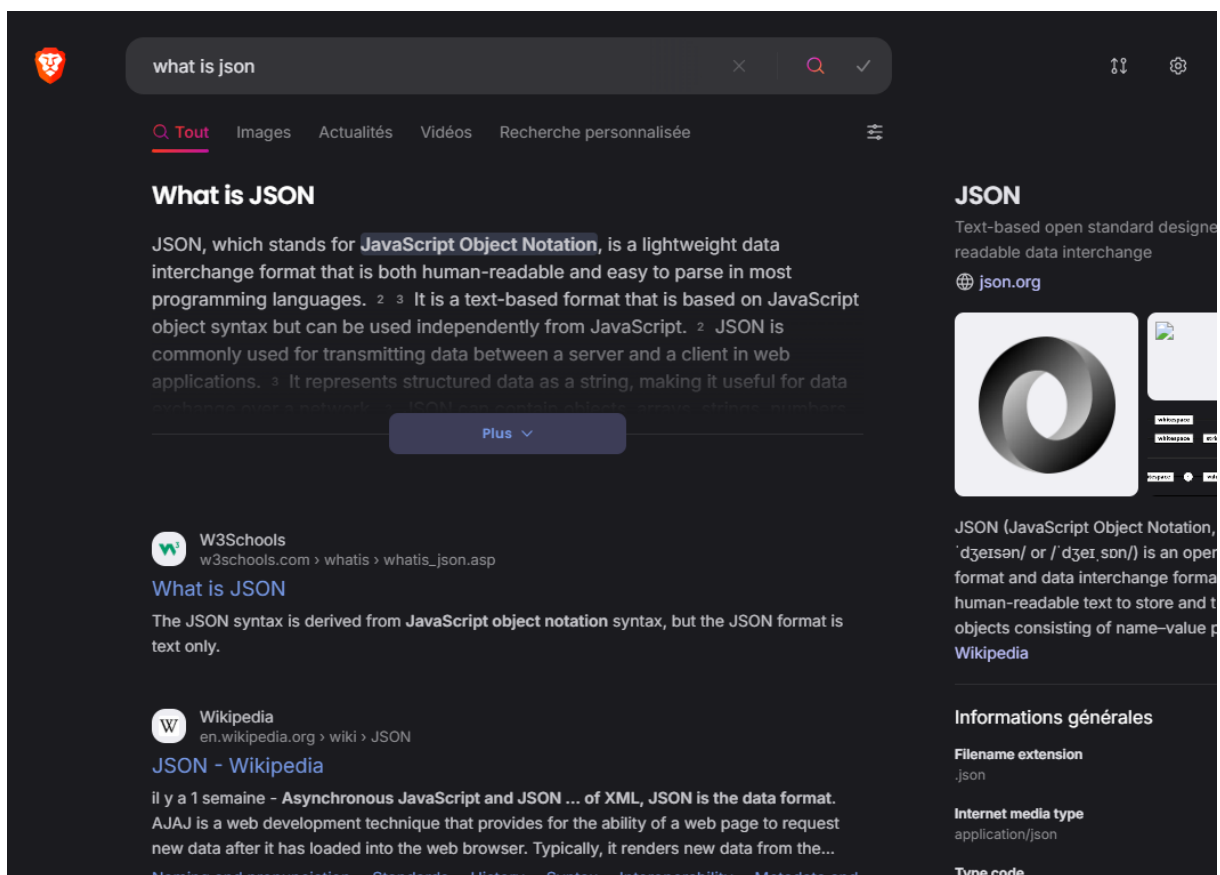
3.9 Recours à l'intelligence artificielle

Durant ce projet j'ai eu recours à l'intelligence artificielle, j'en ai principalement utilisé deux différentes : ChatGPT et Leo AI.

J'utilise Brave comme navigateur et moteur de recherche, c'est là où j'ai eu recours à Leo AI qui est intégrée au moteur de recherche.

Mon utilisation de Leo AI se restreint à du résumé de site web, document, PDF.

Lors d'une recherche sur Brave (moteur de recherche), Leo génère un résumé d'un des premiers sites et ce résumé est placé en haut de la page avant tout lien.



30 Exemple Leo IA

Ça m'est arrivé d'utiliser Leo IA pour trouver une information rapidement même si en règle générale je vais chercher sur sites proposés.

Parfois quand je ne trouvais pas l'information que je cherchais, j'allais consulter ce que Leo proposait et certaine fois cela m'a permis de gagner du temps et d'éviter de me perdre sur la 10ème page de liens.

J'ai également eu recours à ChatGPT pour plusieurs choses.

Premièrement il m'a permis de créer un logo et une icône simple sans y perdre trop de temps. Ce logo et icône ont également été fait en blanc.

Pour la génération d'image il m'a aussi servi pour mes premières maquettes.

ChatGPT m'a aussi été utile pour accélérer mes recherches.

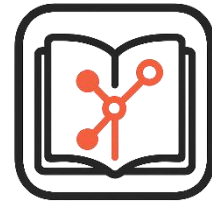
Par exemple pour l'export du JDT en PDF, j'ai utilisé ChatGPT pour me résumer les différents moyens pour arriver à mon but. Malheureusement, ça ne m'a pas empêché de partir sur une certaine librairie et après de multiples bugs et problème de compatibilité, j'ai changé de librairie.

Je l'ai également utilisé pour me générer de fausses données pour des test (Json pour le mettre dans le fichier gitj car à ce moment je n'avais pas encore l'import depuis GitHub).

Mais où il a été le plus utile est pour les petits bug, quand j'avais un problème que je n'arrivais pas à résoudre ou comment trouver la solution en ligne, je passais par ChatGPT pour gagner du temps et surtout pour pas en gâcher. Certaine fois quand je n'avais pas beaucoup d'avance, j'utilisais ChatGPT sans faire trop de recherche au préalable.



31 GitJournal Logo



32 Icone GitJournal

3.10 Liste des documents fournis

Liste des documents fournis :

- Rapport
- Journal de Travail

4 Conclusions

4.1 Objectifs atteint et non-atteints

Des 6 stories, 1 n'a pas été validée et 2 n'ont pas été entièrement complétées.

Les stories complétées sont :

[Donner son Token à l'app](#)
[Reprendre mon JDT sur un autre poste](#)
[Changer de Repository et d'utilisateur](#)

La story non validée est :

[Afficher mes commits sous forme de JDT](#)

Il manque la prise en charge du Markdown ce qui est une feature plutôt importante ce qui invalide cette story

Les 2 stories dont tous les tests n'ont pas été validé sont :

[Gérer les entrées](#)

Il manque le déplacement entre les cases via TAB et Shift+TAB. Cela est beaucoup moins handicapant que le manque du Markdown. Cette story a quand même été validée.

[Exporter en PDF](#)

Sur le fichier le total d'heure n'apparaît pas et il y a parfois des bouts de tableau seul. Cette story a quand même été validée.

4.2 Bilan Personnel

Personnellement j'ai particulièrement apprécié ce projet. J'avais déjà utilisé l'API de GitHub et le WPF mais je n'avais jamais utilisé de librairie pour générer des PDFs. Ce projet est un projet concret qui va potentiellement être utilisé pour générer des JDT ce qui rend le projet encore plus concret et motivant.

4.3 Difficultés

J'ai rencontré 2 difficultés, bien que pas si graves.

La première est la gestion de l'affichage du JDT dans GitJournal.
En premier, j'avais réussi à tout afficher mais en définissant la taille des colonnes et les longs textes n'étaient pas supportés.

Après un peu de travail je suis arrivé à gérer automatiquement la taille des colonnes de manière suffisante, néanmoins cela peut encore être amélioré.

Pour la gestion des longues lignes, j'ai dû rajouter une option qui permet de gérer l'affichage de la case multiligne et de toutes les case de la même ligne.

La deuxième difficulté est encore un fois liée à l'affichage du JDT mais cette fois dans le fichier PDF.

Contrairement à l'affichage dans l'app, la gestion des colonnes était plutôt simple car simplifiée par le package que j'ai utilisé (PdfSharpCore).

En revanche, la gestion du multiligne a été un casse-tête.

La première itération de la génération paraissait bien jusqu'au moment où du contenu étant plus long que la case apparaît. Il dépassait de la case se mêlait au texte de la case suivante ce qui rendait les deux textes illisibles.

J'ai donc dû trouver un moyen pour calculer la longueur que le texte allait prendre et s'il avait besoin de plus de longueur que la longueur de la case moins les marges, alors retour à la ligne.

Cela a apporté un autre problème, l'affichage vertical car maintenant toutes les case ne font pas la même hauteur ce qui a causé des cases de se chevaucher. Ce n'était pas si dur à fixer heureusement.

Dans le produit final, les mots très longs ne sont pas gérés, il est donc possible qu'il dépasse de la case. Durant les tests où j'ai découvert ce comportement, un lien était le "mot" trop long.

4.4 Améliorations possibles

Ajout entrée

Une fois le JDT display, on peut ajouter des nouvelles entrées, sur la page d'ajout il serait bien de supprimer le champ " Utilisateur " et que ça soit automatiquement l'utilisateur connecté.

Tab dans le JDT

Quand on navigue dans le JDT et que l'on modifie les champs, la touche tab ne fait rien. Il faudrait que la touche tab permette de passer de case en case et que shift tab fasse l'inverse.

Filtre temporel

Quand on génère le JDT, GitJournal récupère tous les commits en partant du premier.

Une feature intéressante serait de pouvoir choisir la date à laquelle le JDT commence. Cela permettrait plus de flexibilité à l'utilisateur.

Contenu du PDF

Sur le PDF, les bordures sont un petit peu trop large, il faudrait les rendre plus fines.

Génération du PDF

Lors de l'export du Journal en PDF, une fois le PDF complètement généré il n'y a aucun popup ou tout autre moyen de savoir quand et si le fichier a bien été généré. Il faudrait un moyen de le savoir. Un simple popup comme avec le PAT est suffisant.

Dans le l'entête du PDF il y a juste "Journal de travail" écrit. Quant au pied de page il y a la date, le propriétaire du repository et le nom du fichier.

Il faudrait que l'utilisateur ait un control sur ces informations. Une simple de page avec les valeurs des différents champs serait le mieux à mon avis.

Fichier gitj

Quand un essaye d'ouvrir un fichier gitj avec GitJournal, rien ne se passe.

Ça serait bien que cela permette d'ouvrir le fichier gitj. Si l'identifiant est stocké, il serait connecté automatiquement comme au démarrage de l'app.

Gestion PAT

Lors premier lancement, la première chose à faire est de donner son PAT à l'app. La gestion d'erreur est bien faite mais si on n'a pas de réseau, GitJournal nous dit que le PAT est invalide. Il faudrait qu'il indique que GitHub n'est pas joignable et/ou que l'on a pas de connexion internet.

Manager les commits supprimés

Pour l'instant, on peut ajouter, supprimer et modifier les entrées (commits) dans le JDT.

Quand on supprime l'entrée dans le JDT, on ne peut pas la revenir en arrière. Permettre à l'utilisateur de visualiser ces commits supprimés et de les rerajouter au JDT serais un bon ajout très pratique. Ils s'afficheraient comme tous les autre commits dans la console mais ni en jaune ni en vert mais en rouge. Quand ces commits supprimés sont affichés, un bouton permettant d'annuler la suppression des commits sélectionnés apparaîtrait. Même si ces commits sont affichées dans la console, ils ne seront pas ajoutés au JDT exporté en PDF.

5 Annexes

5.1 Résumé du rapport du TPI / version succincte de la documentation

5.1.1 Situation de départ

GitJournal a pour but de simplifier le fait de tenir un JDT et de gagner du temps. Cela demande un peu discipline mais si bien fait permet de générer un Journal De Travail en se basant sur les commits GitHub. On peut également apporter des modifications si besoin, il est possible de partager cette version modifiée et de la lire depuis ou on veut via GitJournal. Finalement, GitJournal génère un PDF incluant les modifications apportées par l'utilisateur.

Le code est disponible sur GitHub : <https://github.com/Morgan-DD/GitJournal>.

5.1.2 Mise en œuvre

Ce projet a été découpé en 2 sprints durant approximativement 2 semaines chacun.

J'ai passé la première partie du projet à planifier ces sprints et à écrire les user stories. Cela m'a permis de ne pas m'égarer par la suite.

Ensuite, j'ai réalisé les interfaces en suivant ce que j'avais définis lors de l'écriture des user stories. J'ai utilisé des UserControl pour faciliter l'affichage.

Au niveau de la logique, je me suis occupé du "login" via le PAT en premier. J'avais déjà manipulé le PAT lors de mon Pré TPI donc cela a été simple et rapide.

Après cela, je suis parti sur le choix du repository et utilisateur pour l'affichage du JDT. J'ai commis une erreur ici que j'ai lors de la première sprint review. Pas tous les repositories étaient affichés dû à de la pagination du coté de GitHub.

Ensuite, ce fut le tour à l'affichage du JDT dans l'application. J'ai vraiment trouvé cette partie intéressante bien que complexe. J'ai aussi dû passer beaucoup de temps sur le traitement des données pour savoir lesquels afficher et prendre en compte les filtres. J'ai eu le même problème que lors de l'affichage des repositories, pas tous les commits ne s'affichaient de nouveau dû à de la pagination.

Finalement je me suis occupé de la génération du PDF. En premier, j'étais parti sur une librairie que j'ai finalement abandonnée car elle ne convenait dans mon cas de figure. Hormis ce contre temp, cette partie c'est bien passé surtout en sachant que je n'avais jamais fait ça avant.

5.1.3 Résultat final

Le produit final est utilisable et fonctionnel bien qu'il manque quelque petites features. Il y a également plusieurs améliorations possibles.

5.2 Sources – Bibliographie

[FlatIcon \(https://www.flaticon.com/\)](https://www.flaticon.com/) :

Banque d'icone gratuites.

Je l'ai utilisé pour des icones pour l'app et pour certains Schémas.

[GitHub \(https://github.com/\)](https://github.com/) :

Je l'ai utilisé pour faire du versionning, écrire mes user stories et pour les release.

Repository de ce projet : <https://github.com/Morgan-DD/GitJournal>

Leo sur [Brave \(https://search.brave.com/\)](https://search.brave.com/) :

Je l'ai utilisé pour m'aider dans mes recherches. Voir [3.9 Recours à l'intelligence artificielle](#) pour plus d'information.

[ChatGPT \(https://chatgpt.com/\)](https://chatgpt.com/) :

Je l'ai utilisé pour quand j'étais bloqué ou pour m'aider à choisir par exemple la librairie pour la génération de PDF. Voir [3.9 Recours à l'intelligence artificielle](#) pour plus d'information.

[Stackoverflow \(https://stackoverflow.com/questions\)](https://stackoverflow.com/questions) :

Forum où je me retrouvais souvent pour trouver une réponse à ma question.

[pdfsharp \(https://www.pdfsharp.net/\)](https://www.pdfsharp.net/) :

Site de la librairie que j'ai utilisée pour générer les PDF.

5.3 Journal de travail

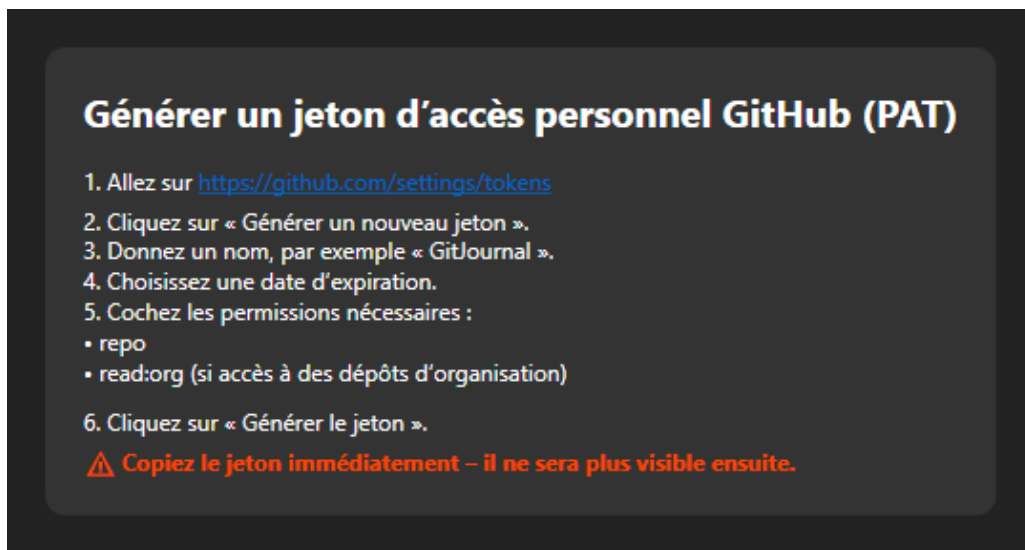
Le journal de travail se trouve en annexe. Il est disponible dans le repository dans le dossier livrable.

5.4 Manuel d'Utilisation

5.4.1 Génération PAT

La génération du PAT n'est pas forcément simple. Il faut également choisir les bons paramètres pour que le token donne les bons droits à GitJournal.

Dans l'app directement, en haut dans la barre il y a un champ Aide > PAT.
Une fenêtre d'aide pour la génération de PAT apparaît :



33 Fenêtre guide PAT dans GitJournal

5.5 Glossaire

PAT	personal access token, token permettant à accéder au GitHub repository privé de la personne l'ayant généré
UserControl	Les user control sont un type d'objet dans visual studio. On peut les modifier et leurs donner l'apparence que l'on veut et ensuite les utiliser comme des control comme un bouton ou un label.